# Calendar Matching

AlgoExpert · Very Hard · Arrays

# The problem

- Two people's meetings ("HH:MM" pairs) and daily bounds; return all free slots of at least `duration` minutes available to both.
- Convert to minutes, merge busy intervals, invert.

# Intuition

- Merge both calendars, including bounds as meetings, then gaps of the right length are free slots.

# Brute force vs optimal

- Check each minute of the day → O(1440)
- Merge sorted busy intervals → O((n+m) log)
- Bounds become busy blocks

# Approach

- 1. Convert times to minutes
- 2. Add each person's bounds as blocked time
- 3. Sort and merge meetings
- 4. Return gaps at least duration long

# Worked example

```
cal1: 9:00–10:30, 12:00–13:00, 16:00–18:00 (bounds 9–20)

cal2: 10:00–11:30, 12:30–14:30, 14:30–15:00, 16:00–17:00 (bounds 10–18:30)
```

Two calendars with daily bounds

# Worked example

```
sort all busy intervals by start

merge overlaps: 0–10:00, 10:00–11:30, 12:00–15:00, 16:00–18:00, 18:30–24:00
```

Merge all busy intervals

# Worked example

```
gaps ≥ 30 min: 11:30–12:00, 15:00–16:00, 18:00–18:30
```

Free gaps long enough

# Java solution (1/2)

```java
public class CalendarMatching {
    public static List<String[]> calendarMatching(List<String[]> cal1, String[] bounds1, List<String[]> cal2, String[] bounds2, int duration) {
        List<int[]> busy = new ArrayList<>();
        addCal(busy, cal1, bounds1);
        addCal(busy, cal2, bounds2);
        busy.sort((a, b) -> Integer.compare(a[0], b[0]));
        List<int[]> merged = new ArrayList<>();
        for (int[] b : busy) {
            if (!merged.isEmpty() && merged.get(merged.size() - 1)[1] >= b[0])
                merged.get(merged.size() - 1)[1] = Math.max(merged.get(merged.size() - 1)[1], b[1]);
            else merged.add(b.clone());
        }
        List<String[]> out = new ArrayList<>();
        for (int i = 1; i < merged.size(); i++) {
            int s = merged.get(i - 1)[1], e = merged.get(i)[0];
            if (e - s >= duration) out.add(new String[]{fmt(s), fmt(e)});
        }
        return out;
    }
```

# Java solution (2/2)

```
    private static void addCal(List<int[]> busy, List<String[]> cal, String[] bounds) {
        busy.add(new int[]{0, min(bounds[0])});
        for (String[] m : cal) busy.add(new int[]{min(m[0]), min(m[1])});
        busy.add(new int[]{min(bounds[1]), 24 * 60});
    }

    private static int min(String t) { String[] p = t.split(":"); return Integer.parseInt(p[0]) * 60 + Integer.parseInt(p[1]); }

    private static String fmt(int m) { return (m / 60) + ":" + String.format("%02d", m % 60); }
}
```

# Complexity

- Time: O(c1 + c2)
- Space: O(c1 + c2)

# Edge cases & pitfalls

- Convert times to minutes
- Back-to-back meetings merge
- Gap must be ≥ duration

# Interview tips & follow-ups

- Generalises to k calendars

# Recap

- Merge both calendars, including bounds as meetings, then gaps of the right length are free slots.
- Complexity: O(c1 + c2)